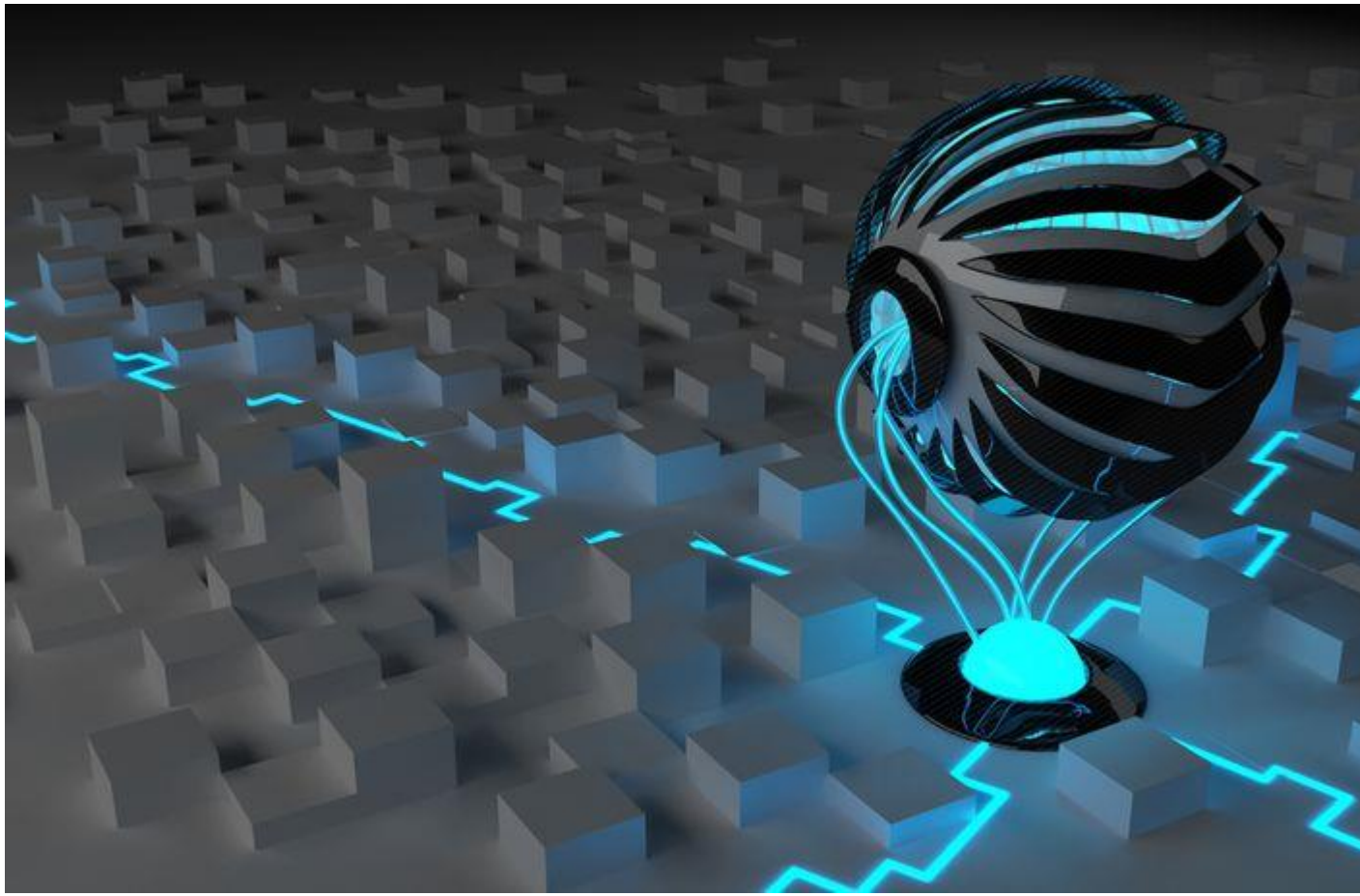
Computer Graphics and Visualization

Chapter 5

Visible Surface Detection



Prepared by:

Asst. Prof. Sanjivan Satyal

Visible Surface Determination [4hours]

- Image Space and Object Space techniques
- Back Face Detection
- Z-Buffer
- A-Buffer
- Scan-Line method

Visible Surface Detection (Hidden Surface Removal) Method

- Hidden surface removal or visible surface determination is the process used to determine which surfaces and parts of surfaces are not visible from a certain viewpoint
- It is necessary to render an image correctly, so that one may not view features hidden behind the model itself, allowing only the naturally viewable portion of the graphics to be visible
- To identify those parts of a scene that are visible from a chosen viewing position (**visible-surface detection methods**)
- Surfaces which are obscured by other opaque (solid) surfaces along the line of sight are invisible to the viewer so can be eliminated (**hidden-surface elimination methods**)
- Once the visible, surfaces are identified surface rendering techniques are applied on them to obscure the hidden surfaces

- **Characteristics of approaches:**

  - ❑Require large memory size?
  - ❑Require long processing time?
  - ❑Applicable to which types of objects?

- **Considerations:**

  - ❑Complexity of the scene
  - ❑Type of objects in the scene
  - ❑Available equipment
  - ❑Static or animated?

Visible Surface Determination

- Visible surface detection methods are broadly **classified** according to whether they deal with objects or with their projected images.
  1. **Object-Space methods (OSM)**
  2. **Image-Space methods (ISM)**

1. Object-Space methods(OSM):

- Deal with object definition directly
- Compares objects and parts of objects to each other within the scene definition to determine which surface as a whole we should label as visible
- E.g. Back-face detection method, Line Display algorithms in wire-frame display

2. Image-Space methods(ISM):

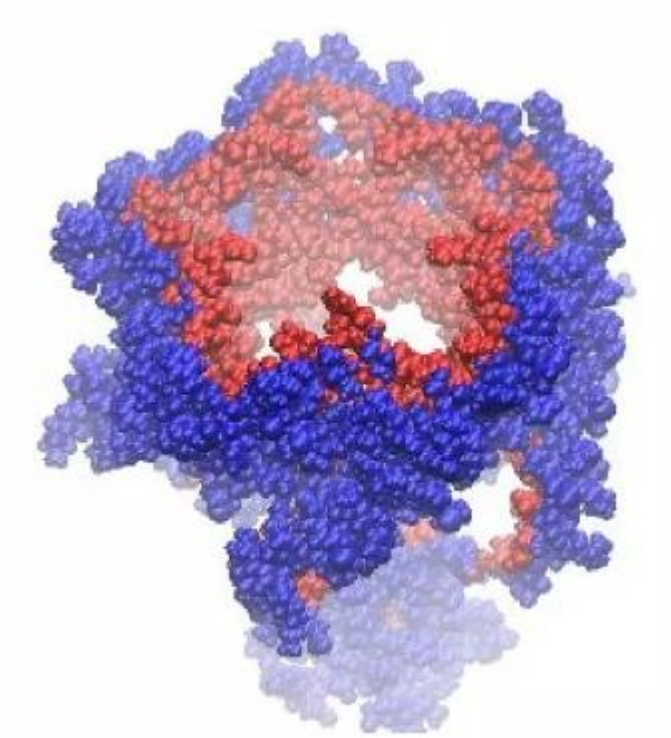
- ❑Deals with projected image of an object
- ❑Visibility is decided point by point at each pixel position on the projection plane
- ❑E.g. Depth-buffer method, Scan-line method, Area-subdivision method

- Most visible surface detection algorithm use image-space-method but in some cases object space methods are also used for it

Depth Cueing

- Depth cueing is implemented by having objects blend into the background color with increasing distance from the viewer.
  - The range of distances over which this blending occurs is controlled by the sliders
- To create realistic image, the depth information is important so that we can easily identify, for a particular viewing direction, which is the front and which is the back of displayed objects
- The depth of an object can be represented by the **intensity** of the image
- The parts of the objects closest to the viewing position are displayed with the highest intensities and objects farther away are displayed with decreasing intensities. This effect is known as '**depth cueing**'

- Hidden surfaces are not removed but displayed with different effects such as intensity, color, or shadow for giving hint for third dimension of the object.



Back-Face Detection Method(Plane equation Method)

- A fast and simple **object-space method** for identifying the back faces of a polyhedron
- It is based on the performing inside-outside test

First Method:

- A point (x, y, z) is "**inside**" a polygon surface with plane parameters A, B, C, and D if **Ax+By+Cz+D < 0** (from plane equation)
- When an inside point is along the line of sight to the surface, the polygon must be a back face

Alternatively

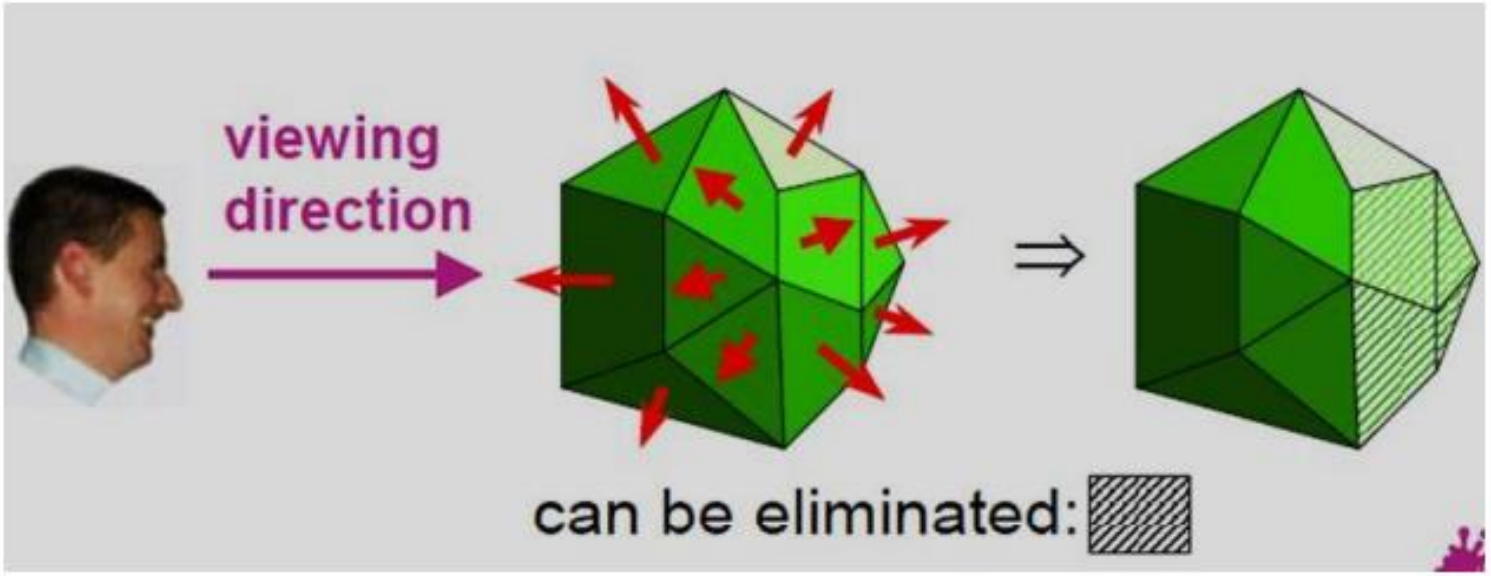
1. Make the centre of object at origin
2. Calculate the value of D of the plane by taking three points in anticlockwise direction. In eq. **Ax+By+Cz+D = 0**
3. If **D<=0** the surface is **visible**

   D>0 the surface in **invisible**

if A,B,C remain constant , then varying value of D result in a whole family of parallel plane

- if **D>0,** plane is **behind** the origin (Away from observer)
- if **D<0** , plane is in **front** of origin (toward the observer)

Back-Face Detection Method(Plane equation Method)

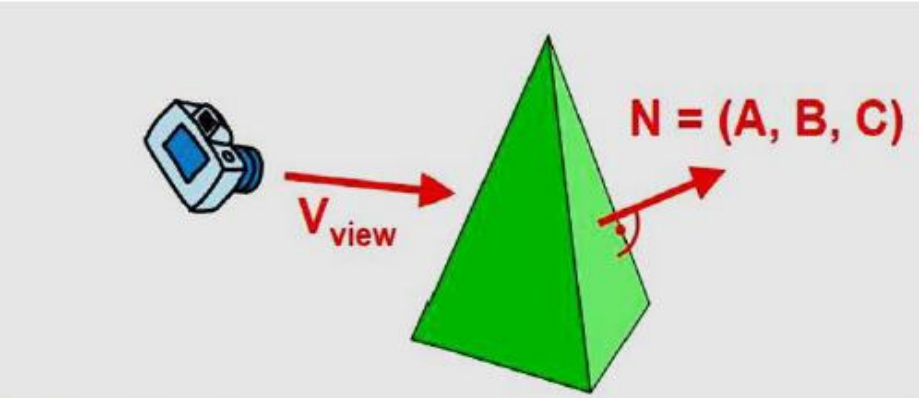


Back-Face Detection Method(Plane equation Method)

Second Method:

- Consider Normal Vector **N** = (A,B,C) to polygon surface and if **V** is a vector in the viewing direction from the eye (or "camera") position

- The polygon is **back face** if: **V.N>0**
- The polygon is **visible Surface** If **V.N<0**
- If object description is converted to projection co-ordinates then, **V** is along z_v axis i.e **V** = $(0,0,V_z)$, then:
  - **V.N** = V_zC
  - i.e V.N > 0 then V_zC
- So we need to consider only sign of C

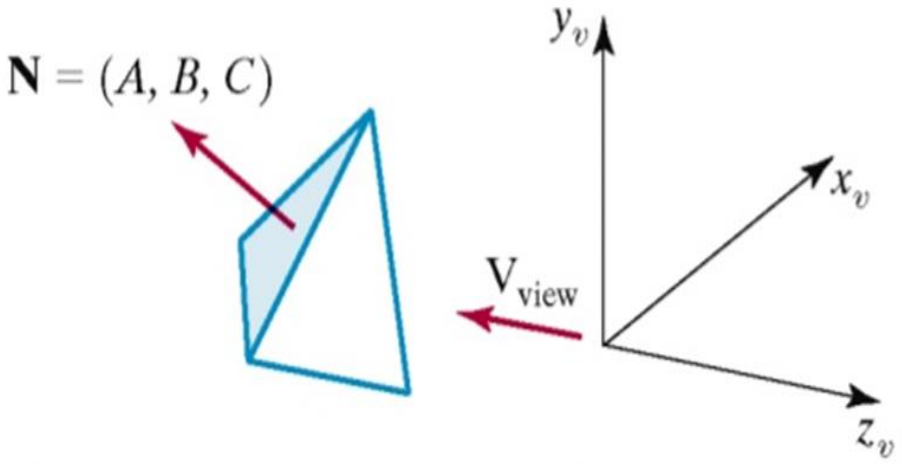


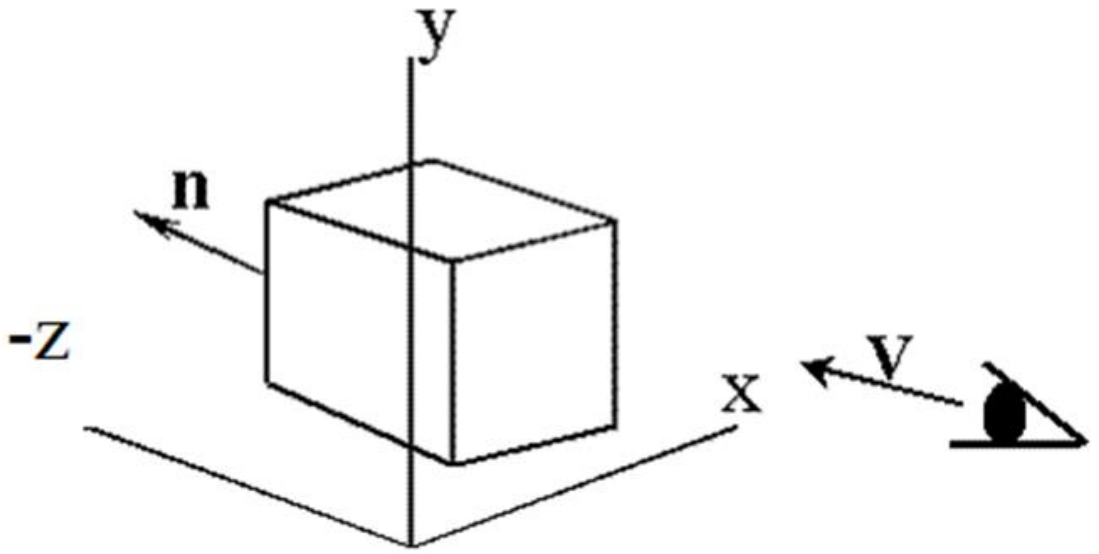
For the right handed viewing system

- if the ‘z’ component of the **normal vector is negative**, then it is **back face**
- If the ‘z’ component of the vector is positive then it is a front face
- In a right-handed viewing system with viewing direction along the negative z axis the polygon is a back face if $C < 0$
- In general, we can label any polygon as a back face if its normal vector has a z component value: **C<=0**

For the left handed viewing system

- If the ‘z’ component of the **normal vector is positive**, then it is **back face**
- If the ‘z’ component of the vector is negative then it is a front face
- Also, back faces have normal vectors that point away from the viewing position and are identified by **C >= 0** when the viewing direction is along the positive z axis





A polygon surface with plane parameter C < 0 in a **right-handed** viewing coordinate system is identified as a back face when the viewing direction is along the **negative z_v axis**.

Doesn't work well for

1. Overlapping front faces due to
   - Multiple objects
   - Concave objects

2. Non Closed Objects

3. Non-polygonal models

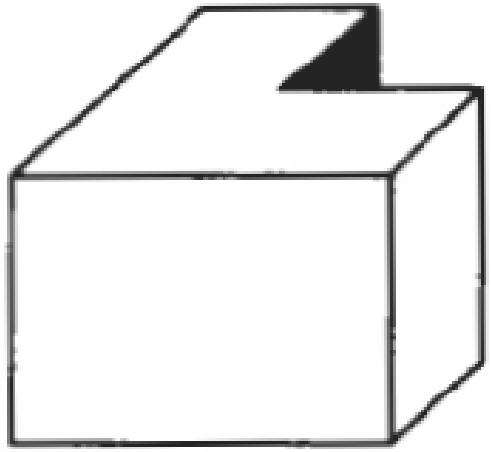
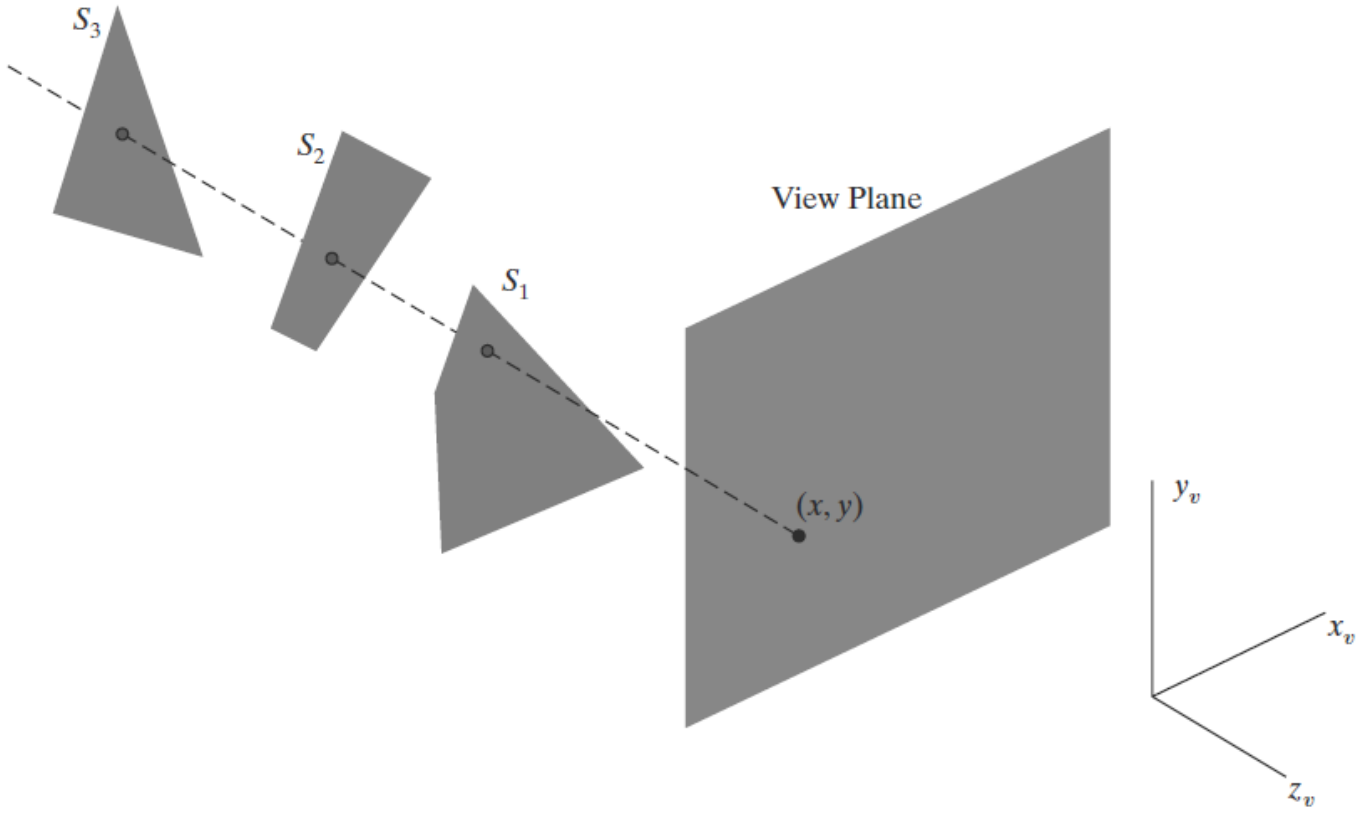


Fig: View of a concave polyhedron with one face partially hidden by other faces

- This method works fine for convex polyhedral, but not necessarily for concave polyhedral or overlapping objects
- So, we need to apply other methods to further determine where the obscured faces are partially or completely hidden by other objects (e.g. Using Depth-Buffer Method or Depth-sort Method)
- This method can only be used on solid objects modelled as a polygon mesh

Depth – Buffer (Z – Buffer Method)

- A commonly used **image-space** approach to detecting visible surfaces is the **depth-buffer method**, which compares surface depths at each pixel position on the projection plane
- This procedure is also referred to as the **z-buffer** method, since object depth is usually measured from the view plane along the z axis of a viewing system.
- Each surface of a scene is processed **separately**, one point at a time across the surface. And each (x, y, z) position on a polygon surface corresponds to the projection point (x, y) on the view plane.



Three surfaces overlapping pixel position (x , y) on the view plane. The visible surface, S_1, has the smallest depth value.

- It is applied very efficiently on surfaces of polygon. Surfaces can be processed in any order. To override the closer polygons from the far ones, two buffers named **frame buffer** and **depth buffer**, are used.
- **Depth buffer (z buffer)** is used to store depth values for x, y position, as surfaces are processed $0 \leq depth \leq 1$.
  - Initially all positions are set to 1 (maximum depth)
- **frame buffer (refresh buffer)** is used to store the intensity value of colour value at each position x, y.
  - Initially **frame buffer** is initialized to background intensity
- The z-coordinates are usually normalized to the range [0, 1].
- so that depth values range from 0 at the near clipping plane (the view plane) to 1.0 at the far clipping plane.

- Each surface listed in the polygon tables is then processed, one scan line at a time, by calculating the depth value at each (x, y) pixel position. This calculated depth is compared to the value previously stored in the depth buffer for that pixel position

- If the calculated depth is **less than** the value stored in the depth buffer, the new depth value is stored. Then the surface color value at that position is computed and placed in the corresponding pixel location in the frame buffer

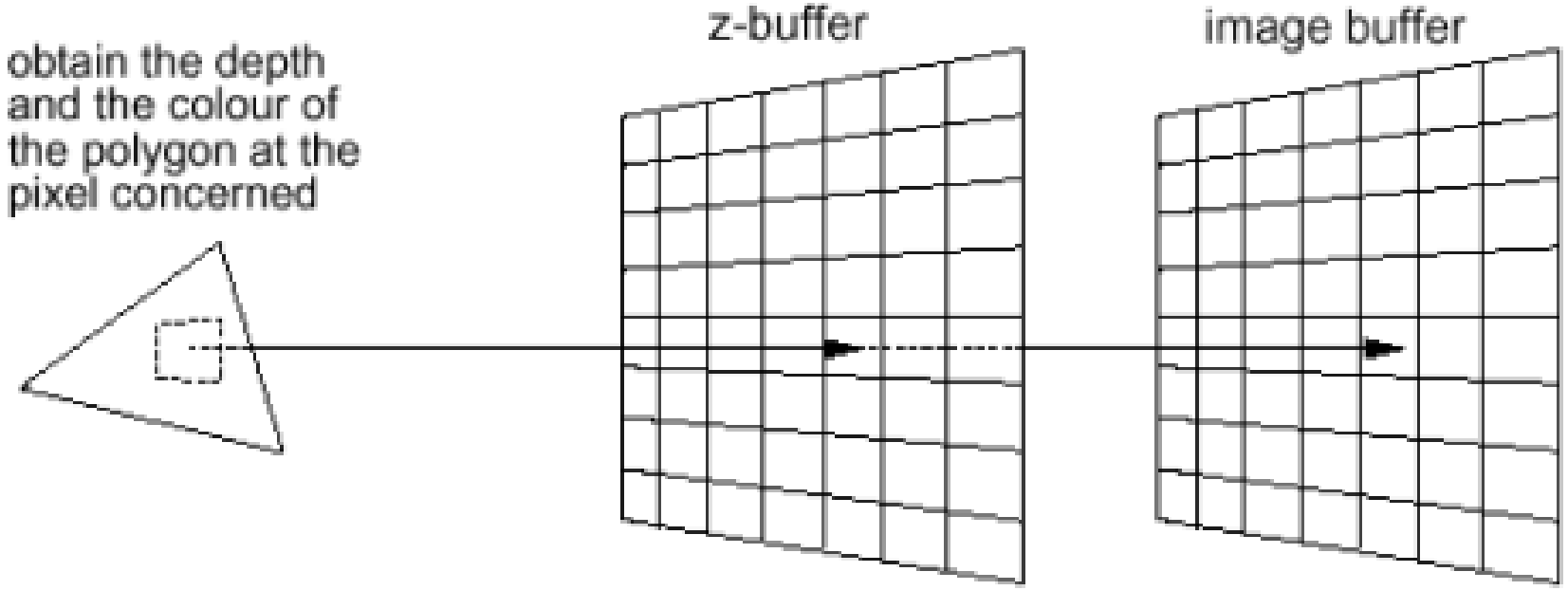
- After all the surfaces have been processed, each pixel of the frame buffer represents the color of a visible surface at that pixel

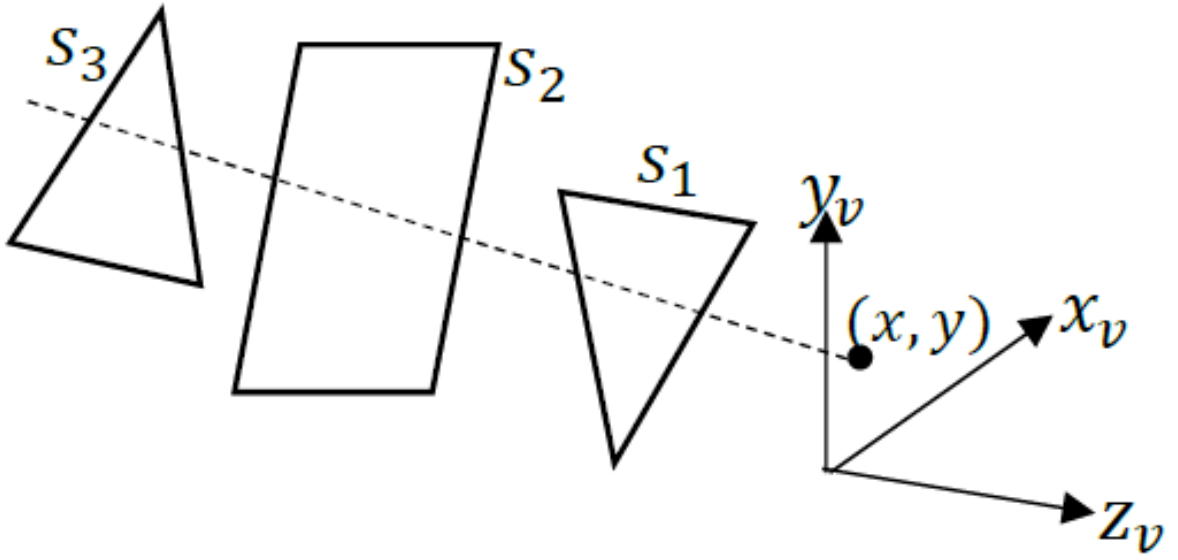
- This algorithm assumes that depth values are normalized on the range from 0.0 to 1.0 with the view plane at depth= 0 and the farthest depth= 1

1) Set the buffer values:
    - DepthBuffer(x, y)=1,
    - FrameBuffer(x, y)= Background color

2) Process each polygon, one at a time as follows:
    - For each projected (x, y) pixel position of a polygon, calculate depth 'z'.
    - If **z< DepthBuffer(x,y),** compute surface color and set
        - DepthBuffer(x, y)=z,
        - FrameBuffer(x, y)=surfacecolor(x, y)

3) After all surface have been processed, Draw object using the depth buffer containing depth values for visible surface and frame buffer containing corresponding color values for those surface. The depth value of the surface position (x,y) is calculated by plane equation of surface.


obtain the depth
and the colour of
the polygon at the
pixel concerned
z-buffer
image buffer


s_3
s_2
s_1
y_v
(x, y)
x_v
z_v

Depth Calculation

- In the figure, at view plane position (x, y), surface $s1$ has the smallest depth from the view plane, so it is visible at that position.
- Depth value for a surface position (x, y) are calculated from the plane equation as;

$$z = \frac{-Ax - By - D}{C}$$

Now, the depth z' of the next position **(x+1, y)** is obtained as:

Let depth z' at (x + 1 , y)

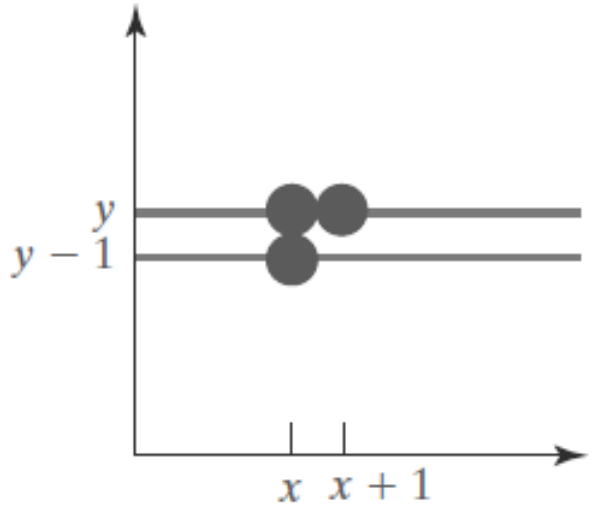
z' = {-A(x+1) – By – D}/c

z' = {-Ax –By – D-A}/c

z' = (-Ax –By – D)/c - A/c

or

$$z' = z - \frac{A}{C}$$



The ratio $-A/C$ is constant for each surface, so succeeding depth values across a scan line are obtained from preceding values with a single addition.

Advantages:

- It reduces the speed problem if implemented in hardware.
- It processes one object at a time
- Simple and easy to implement, does not require additional data structures.
- The z-value of a polygon can be calculated incrementally
- No pre-sorting of polygons is needed.
- No object-object comparison is required.
- Can be applied to non-polygonal objects.
- Hardware implementations of the algorithm are available in some graphics workstation
- For large images, the algorithm could be applied to, eg., the 4 quadrants of the image separately, so as to reduce the requirement of a large additional buffer.

Disadvantages:

- It is a time consuming process as it requires comparison for each pixel instead of for the entire polygon
- Deals only with opaque surface but not with transparent surface i.e. it can only find one visible surface at each pixel position
- This method requires an additional buffer (if compared with the Depth-Sort Method) and the overheads involved in updating the buffer. So this method is less attractive in the cases where only a few objects in the scene are to be rendered
- It requires large memory

A – Buffer Method

- The **A-buffer** (**anti-aliased**, **area-averaged**, **accumulation buffer**) is an extension of the ideas in the **depth-buffer** method (other end of the alphabet from "**z-buffer**").

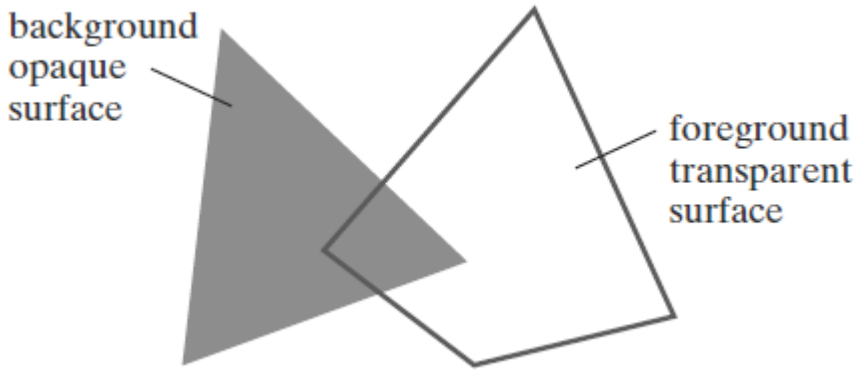
- The **A-buffer** method is an extension of the depth-buffer method.

- The A-buffer method is a visibility detection method developed at Lucas film studio for the surface rendering system REYES (Render Everything You Ever Saw)

- The buffer region for this procedure is referred to as the ***accumulation buffer***, because it is used to store a variety of surface data, in addition to depth values

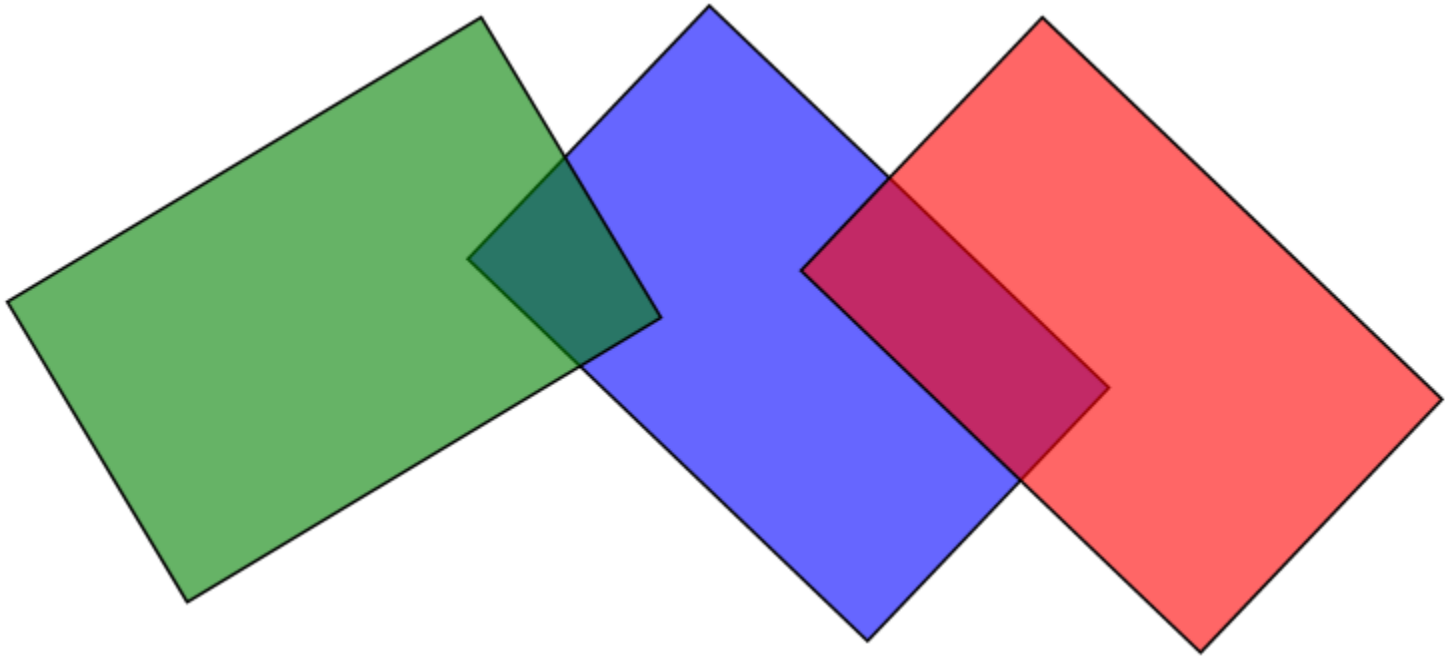
- The **A-buffer** method calculates the surface intensity for **multiple surfaces** at each pixel position, and object edges can be antialiased.

- A drawback of the depth-buffer method is that it identifies only **one** visible surface at each pixel position
  - It deals only with opaque surfaces and cannot accumulate color values for more than one surface, as is necessary transparent surfaces are to be displayed



Viewing an opaque surface through a transparent surface requires multiple color inputs and the application of color-blending operations.

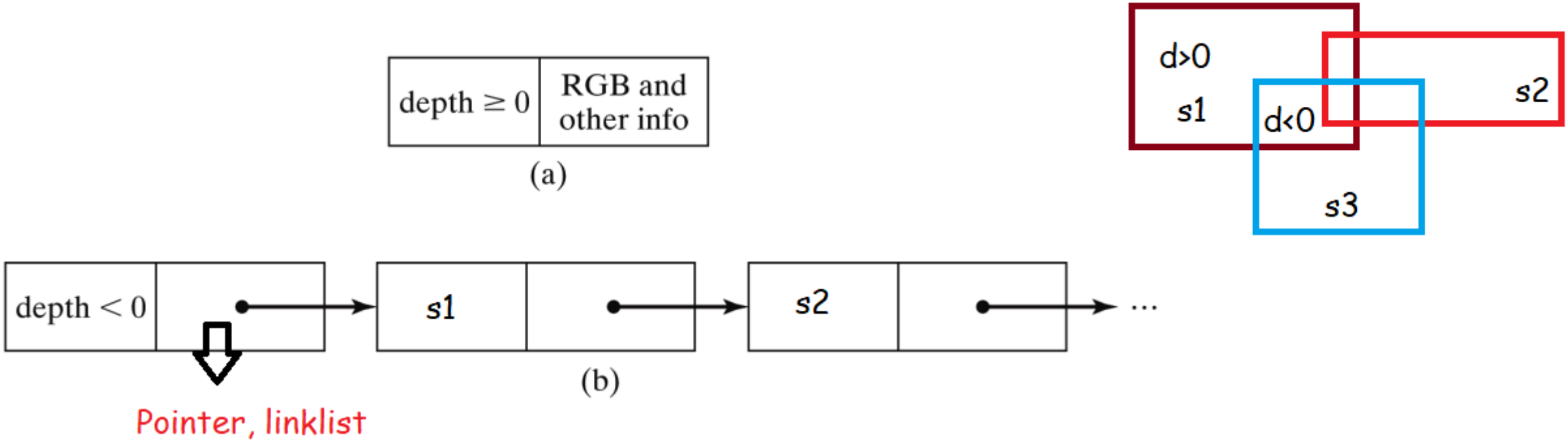
- The **A-buffer** method expands the depth-buffer algorithm so that each position in the buffer can reference a linked list of surfaces
- This allows a pixel color to be computed as a combination of different surface colors for transparency or antialiasing effects



A – Buffer Method…

Each pixel position in the A-Buffer has two fields

- **Depth Field** : Stores a real-number value (positive, negative, or zero)
  - Non Negative: **single** surface contributes to pixel intensity
  - Negative : **multiple** surfaces contribute to pixel intensity
- **Surface data field** or **Intensity Field**: stores surface-intensity information or a pointer value
  - **Surface intensity** if single surface, stores the RGB components of the surface color at that point and percent of pixel coverage
  - if multiple surfaces then the color field stores a pointer to a linked list of surface data



- Two possible organizations for surface information in an A-buffer representation for a pixel position.
  - When a **single** surface overlaps the pixel, the surface depth, color, and other information are stored as in (a).
  - When **more than one** surface overlaps the pixel, a **linked list** of surface data is stored as in (b).

- Surface information in the A-buffer includes

  a. RGB intensity components
  b. Opacity parameter (percent of transparency)
  c. Depth
  d. Percent of area coverage
  e. Surface identifier
  f. Other surface-rendering parameters

- The algorithm proceeds just like the depth buffer algorithm. The depth and opacity values are used to determine the final colour of a pixel
- Scan lines are processed to determine surface overlaps of pixels across the individual scan lines
- Surfaces are subdivided into a polygon mesh and clipped against the pixel boundaries
- Using the opacity factors and percent of surface overlaps, we can calculate the intensity of each pixel as an average of the contributions from the over lapping surfaces.

Scan-Line Method (Study Yourself)

- This **image-space** method for removing hidden surfaces is an extension of the **scan-line algorithm** for filling polygon interiors where, we deal with multiple surfaces rather than one.

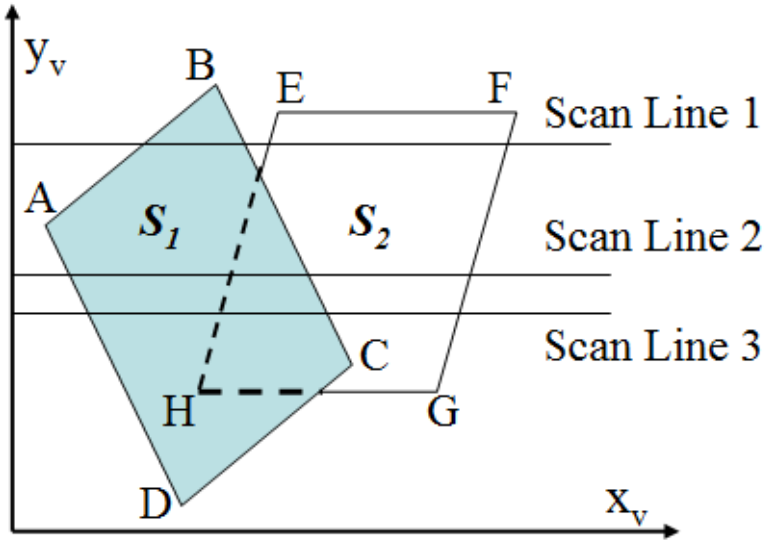


Fig. Scan lines crossing the projection of two surfaces, S_1 and S_2 in the view plane. Dashed lines indicate the boundaries of hidden surfaces.

Scan-Line Method…

- Extension of scan line algorithm for filling polygon interiors
- Instead of filling just one surface, we deal with multiple surfaces
- As each scan line is processed, all polygon surfaces intersecting that line are examined to determine which are visible
  - Across each scan line, depth calculations are made for each overlapping surface to determine, which is nearest to view plane
- When the visible surface has been determined, the intensity value for that position is entered into the frame buffer

Two important tables are maintained:

A. **Edge table** containing

- Coordinate endpoints for each line in a scene
- Inverse slope of each line
- Pointers into polygon table to identify the surfaces bounded by each line

B. **Surface table** containing

- Coefficients of the plane equation for each surface
- surface material properties, other surface data
- Pointers to edge table

- **Active Edge List**
  - To keep a trace of which edges are intersected by the given scan line
  - contains only those edges that cross the current scan line, sorted in order of increasing *x*

  - Define flags for each surface to indicate whether a position is inside or outside the surface

Scan-Line Method…

I. Initialize the necessary data structure

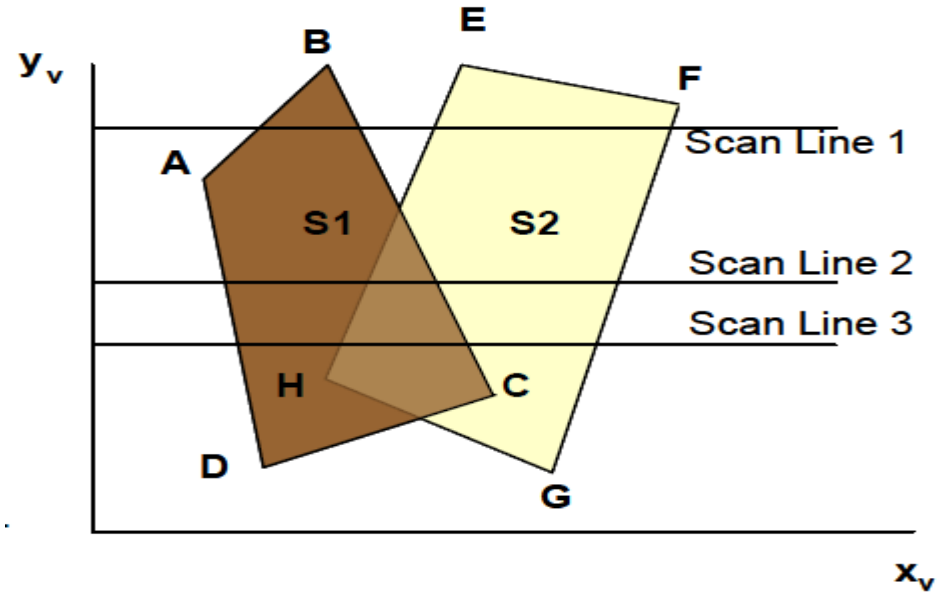
1. Edge table containing end point coordinates, inverse slope and polygon pointer.
2. Surface table containing plane coefficients and surface intensity
3. Active Edge List
4. Flag for each surface

II. For each scan line repeat

1. update active edge list
2. determine point of intersection and set surface on or off.
3. If flag is on, store its value in the frame buffer
4. If more than one surface is on, do depth sorting and store the intensity of surface nearest to view plane in the refresh buffer

- The active list for scan line 1 contains edges AB, BC, EH & FG. Similarly for scan line 2, active list contains edges AD, EH, BC &FG.

	Edge pairs	Flags for	
		S_1	S_2
For scan line 1	AB & BC	on	off
	BC & EH	off	off
	EH & FG	off	on
For scan line 2	AD & EH	on	off
	EH & BC	on	on
	BC & FG	off	on



- Here, between EH & BC, the flags for both surfaces are ON so depth calculation is necessary. But between other edge pairs no depth calculation is necessary.

For scan line 1

- The active edge list contains edges AB,BC,EH, FG
- Between edges AB and BC, only *flags for s1 == on* and
- between edges EH and FG, only *flags for s2==on*
  - no depth calculation needed and corresponding surface intensities are entered in refresh buffer

For scan line 2

- The active edge list contains edges AD,EH,BC and FG
- Between edges AD and EH, only the *flag for surface s1 == on*
- Between edges EH and BC *flags for both surfaces == on*
  - Depth calculation (using plane coefficients) is needed
  - In this example ,say s1 is nearer to the view plane than s2, so intensities for surface s1 are loaded into the refresh buffer until boundary BC is encountered
- Between edges BC and FG flag for s1==off and *flag for s2 == on*
  - Intensities for s2 are loaded on refresh buffer

- **For scan line 3**

  - same active list of edges as scan line 2
  - No changes have occurred in line intersections, so it is again unnecessary to make depth calculations between edges EH and BC.
  - The two surfaces must be in the same orientation as determined on scan line 2, so the colors for surface S_1 can be entered without further depth calculations

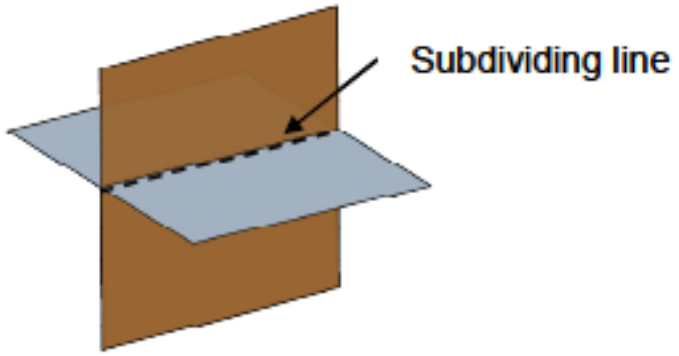
Scan-Line Method…

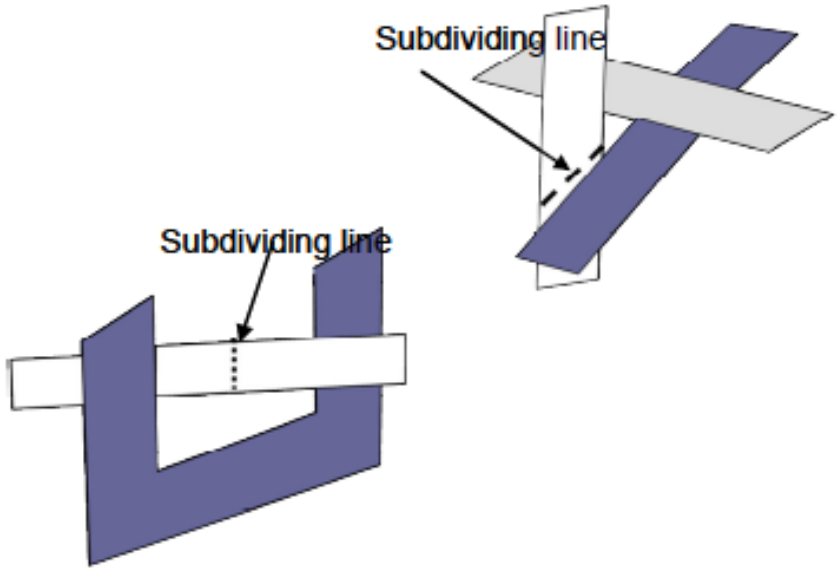
- Any number of overlapping polygon surfaces can be processed with this scanline method.
- Flags for the surfaces are set to indicate whether a position is inside or outside, and depth calculations are performed only at the edges of overlapping surfaces.
- The algorithm is applicable to non-polygonal surfaces
- Memory requirement is less than that for depth-buffer method

Problem:

Dealing with **cut through** surfaces and **cyclic overlap** is problematic when used coherent properties

- Solution: Divide the surface to eliminate the overlap or cut through





Thank You